

WORKSHEET-1

DEEP LEARNING AND APPLICATIONS

VEDAVALLI-V

245HSRDO15

1. Let's say the output of a last fc layer is $[2.0 \ 1.0 \ 0.1]$. If we need to convert this network into predicting class probabilities for 3 classes, how will you go about doing it and what are the probability values for 3 classes?

The output of the last fc layer is the raw scores or logits which are not probability scores. so we should use softmax function as ~~it~~ to we have to classify 3 classes.

Applying softmax to $[2.0, 1.0 \ 0.1]$

Wkt softmax function is $\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}} \dots$

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

$$e^2 = 7.389$$

$$e^1 = 2.718$$

$$e^{0.1} = 1.015$$

Probability values for 3 classes:-

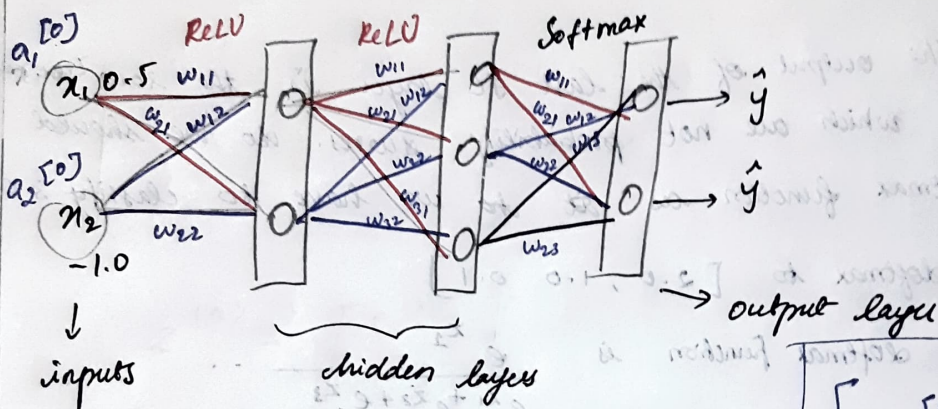
$$\text{Class 1: } \frac{e^2}{e^2 + e^1 + e^{0.1}} = \frac{7.389}{11.212} \approx 0.659$$

$$\text{Class 2: } \frac{e^1}{e^2 + e^1 + e^{0.1}} = \frac{2.718}{11.212} \approx 0.242$$

$$\text{Class 3: } \frac{e^{0.1}}{e^2 + e^1 + e^{0.1}} = \frac{1.015}{11.212} \approx 0.099$$

⇒ Class 1 has the highest probability (65.9%) so the model predicts Class 1.

2. Consider the following FFN. Input $X = [0.5 \ -1.0]$, 2 hidden layers with 2 and 3 neurons respectively. The output layer contains two neurons. ReLU is applied for hidden layers and softmax is applied for the output layer activations. For simplicity assume that there is no bias term connected to neurons. Initialize the weight matrices randomly. Represent activations & weights as vectors and matrices and compute the forward pass output using a series of matrix multiplication.



$$z_1^{[1]} = w_{11}^{[1]} a_1^{[0]} + w_{12}^{[1]} a_2^{[0]}$$

$$z_2^{[1]} = w_{21}^{[1]} a_1^{[0]} + w_{22}^{[1]} a_2^{[0]}$$

$$w^{[1]} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} \end{bmatrix}_{2 \times 2}$$

weight matrix.

Layer 1:

before activation $z^{[1]} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \end{bmatrix}_{2 \times 1}$ after ReLU $a^{[1]} = g(z^{[1]})$

after activation $a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \end{bmatrix}_{2 \times 1}$

Layer 2:

$$z_1^{[2]} = w_{11}^{[2]} a_1^{[1]} + w_{12}^{[2]} a_2^{[1]}$$

$$z_2^{[2]} = w_{21}^{[2]} a_1^{[1]} + w_{22}^{[2]} a_2^{[1]}$$

$$z_3^{[2]} = w_{31}^{[2]} a_1^{[1]} + w_{32}^{[2]} a_2^{[1]}$$

weight matrices $\Rightarrow w^{[2]} = \begin{bmatrix} w_{11}^{[2]} & w_{12}^{[2]} \\ w_{21}^{[2]} & w_{22}^{[2]} \\ w_{31}^{[2]} & w_{32}^{[2]} \end{bmatrix}_{3 \times 2}$

$$w_{(3 \times 2)} \times a_{(2 \times 1)} = z_{(3 \times 1)}$$

$$\begin{bmatrix} z_1^{[2]} \\ z_2^{[2]} \\ z_3^{[2]} \end{bmatrix}_{3 \times 1} \xrightarrow{\text{ReLU}} \begin{bmatrix} a_1^{[2]} \\ a_2^{[2]} \\ a_3^{[2]} \end{bmatrix}_{3 \times 1}$$

Layer 3: (o/p)

$$\begin{aligned} z_1^{[3]} &= w_{11} a_1^{[2]} + w_{12} a_2^{[2]} + w_{13} a_3^{[2]} \\ z_2^{[3]} &= w_{21} a_1^{[2]} + w_{22} a_2^{[2]} + w_{23} a_3^{[2]} \end{aligned} \Rightarrow \begin{bmatrix} z_1^{[3]} \\ z_2^{[3]} \end{bmatrix}_{2 \times 1}$$

weight matrix

$$w^{[3]} = \begin{bmatrix} w_{11}^{[3]} & w_{12}^{[3]} & w_{13}^{[3]} \\ w_{21}^{[3]} & w_{22}^{[3]} & w_{23}^{[3]} \end{bmatrix}_{2 \times 3}$$

$$z_{(2 \times 1)} = w_{(2 \times 3)} \times a_{(3 \times 1)}$$

$$\begin{bmatrix} z_1^{[3]} \\ z_2^{[3]} \end{bmatrix}_{2 \times 1} \xrightarrow{\text{Softmax}} \begin{bmatrix} a_1^{[3]} \\ a_2^{[3]} \end{bmatrix}$$

With example: Random weight initialization

$$w^{[1]} = \begin{bmatrix} 0.2 & -0.4 \\ 0.7 & 0.1 \end{bmatrix}$$

$$w^{[2]} = \begin{bmatrix} 0.6 & -0.1 \\ -0.2 & 0.4 \\ 0.3 & 0.8 \end{bmatrix}$$

$$w^{[3]} = \begin{bmatrix} 0.5 & -0.6 \\ -0.3 & 0.2 \end{bmatrix}$$

$$x = \begin{bmatrix} 0.5 \\ -1 \end{bmatrix}$$

1st layer:

$$z^{(1)} = w^{(1)} a^{(0)}$$

$$= \begin{bmatrix} 0.2 & -0.4 \\ 0.7 & 0.1 \end{bmatrix} \begin{bmatrix} 0.5 \\ -1 \end{bmatrix}$$

$$z_1^{(1)} = 0.2 \times 0.5 + 0.4 = 0.5$$

$$z_2^{(1)} = 0.7 \times 0.5 + 0.1 \times (-1) = 0.25$$

$$z^{(1)} = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}$$

$$a^{(1)} = \text{ReLU}(z^{(1)}) = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}$$

2nd layer:

$$z^{(2)} = W^{(2)} a^{(1)} = \begin{bmatrix} 0.6 & -0.1 \\ -0.2 & 0.4 \\ 0.3 & 0.8 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}$$

$$z_1^{(2)} = 0.6 \times 0.5 + (-0.1) \times 0.25 = 0.275$$

$$z_2^{(2)} = -0.2 \times 0.5 + 0.4 \times 0.25 = 0.0$$

$$z_3^{(2)} = 0.3 \times 0.5 + 0.8 \times 0.25 = 0.15 + 0.20 = 0.35$$

$$z^{(2)} = \begin{bmatrix} 0.275 \\ 0.0 \\ 0.35 \end{bmatrix}$$

$$a^{(2)} = \text{ReLU}(z^{(2)}) \Rightarrow \begin{bmatrix} 0.275 \\ 0 \\ 0.35 \end{bmatrix}$$

Output layer:

$$z^{(3)} = W^{(3)} a^{(2)} = \begin{bmatrix} 0.5 & -0.6 & 0.1 \\ -0.3 & 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 0.275 \\ 0 \\ 0.35 \end{bmatrix}$$

$$z_1^{(3)} = 0.5 \times 0.275 + 0 + 0.1 \times 0.35 = 0.1725$$

$$z_2^{(3)} = -0.3 \times 0.275 + 0 + 0.4 \times 0.35 = 0.0575$$

$$z^{(3)} = \begin{bmatrix} 0.1725 \\ 0.0575 \end{bmatrix}$$

Softmax $\rightarrow z^{(3)}$

$$e^{0.1725} = 1.1882$$

$$e^{0.0575} = 1.0592$$

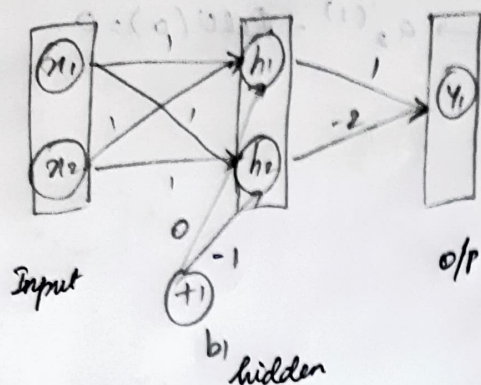
$$e^{0.1725} + e^{0.0575} = 1.1882 + 1.0592 = 2.2474$$

$$\text{Probability of class 1} = \frac{1.1882}{2.2474} = 0.5287$$

$$\text{Probability of class 2} = \frac{1.0592}{2.2474} = 0.4713$$

Predicted class = 0.527.

3. Construct a computational graph, compute the gradients and ^{update} weights for a single forward and backward pass. The activations are ReLU for hidden layers. The input x is shown in the Table below. Use all 4 samples as a single batch.



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

weight matrix

$$W^{(1)} = \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix}_{2 \times 2}$$

Sample 1:

Forward pass

$$z_1^{(1)} = w_{11}^{(1)} a_1^{(0)} + w_{12}^{(1)} a_2^{(0)} + b_1$$

$$z_2^{(1)} = w_{21}^{(1)} a_1^{(0)} + w_{22}^{(1)} a_2^{(0)} + b_2$$

$$z_1^{(1)} = 1 \times 0 + 1 \times 0 = 0 + 0 = 0$$

$$z_2^{(1)} = 1 \times 0 + 1 \times 0 = 0 + 1 = -1$$

$$\text{Relu}(z_1^{(1)}) = \text{Relu}(0) = 0 \quad a_1^{(1)}$$

$$\text{Relu}(z_2^{(1)}) = \text{Relu}(-1) = 0 \quad a_2^{(1)}$$

$$y = w_{11} a_1^{(1)} + w_{12} a_2^{(1)}$$

$$\hat{y} = 1 \times 0 + -2 \times 0 = 0$$

$$\text{loss} = \frac{1}{2} (0 - 0) = 0$$

sample 2: $x_1 = 0$
 $x_2 = 1$

$$z_1^{(1)} = 1 \times 0 + 1 \times 1 + 0 = 1 \rightarrow a_1^{(1)} = \text{ReLU}(1) = 1$$

$$z_2^{(1)} = 1 \times 0 + 1 \times 1 - 1 = 0 \rightarrow a_2^{(1)} = \text{ReLU}(0) = 0.$$

$$y = 1 \times 1 + (-2) \times 0 = 1$$

$$\text{loss} = \frac{1}{2} (1 - 1)^2 = 0.$$

sample 3: $x_1 = 1$
 $x_2 = 0$

$$z_1^{(1)} = 1 \times 1 + 1 \times 0 + 0 = 1 \rightarrow a_1^{(1)} = \text{ReLU}(1) = 1$$

$$z_2^{(1)} = 1 \times 1 + 1 \times 0 - 1 = 0 \rightarrow a_2^{(1)} = \text{ReLU}(0) = 0.$$

$$y = 1 \times 1 + (-2) \times 0 = 1$$

$$\text{loss} = \frac{1}{2} (1 - 1)^2 = 0$$

sample 4: $x_1 = 1$
 $x_2 = 1$

$$z_1^{(1)} = 1 \times 1 + 1 \times 1 + 0 = 2 \rightarrow a_1^{(1)} = \text{ReLU}(2) = 2$$

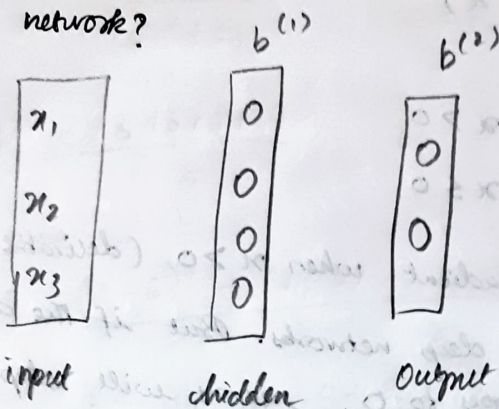
$$z_2^{(1)} = 1 \times 1 + 1 \times 1 + (-1) = 1 \rightarrow a_2^{(1)} = \text{ReLU}(1) = 1$$

$$y = 1 \times 2 + (-2) \times 1 = 0$$

$$\text{loss} = \frac{1}{2} (2 - 0)^2 = 2.$$

We can see that the single forward pass produces zero loss, hence the backward pass yields zero gradients $\rightarrow$ no weight updates and for the so across the batch.

4. Consider a 2-layer fully connected feedforward network with 3 neurons in the input layer, 4 neurons in the hidden layer and 2 neurons in the output layer. Bias terms are connected to all neurons in the hidden and output layers. What will be the total no. of parameters in the network?



$$\begin{array}{rcl}
 3 \times 4 = 12 & + & 4 \times 2 = 8 \\
 \underline{4} & & \underline{2} \\
 16 & & 10 \\
 \downarrow & & \downarrow \\
 \text{Parameters for layer 1} & & \text{Parameters for layer 2}
 \end{array}$$

$$\text{Total parameters} = 26$$

5. Explain vanishing and exploding gradient problems by comparing sigmoid, tanh and ReLU activation functions.

When training the deep neural networks with backpropagation, there is a chance of vanishing gradients and exploding gradients, because gradients shrink or grow layer by layer because we multiply many derivatives of activation function.

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\text{Range} = (0, 1)$$

$$\text{derivative of } \sigma(x) \Rightarrow \sigma(x)(1 - \sigma(x))$$

The maximum value the derivative can have is 0.25 because the range $(0, 0.25)$. Gradients will be less than 0.25 ^{or equal to} do multiply them (small nos) across multiple layers will make the gradients vanish.

The sigmoid saturates for large $|x| \rightarrow$ derivatives tends to 0. Therefore the neurons doesn't learn anything (dead neurons) as we go deeper. The ^{gradient} explosion rarely happens since derivatives are small.

~~tanh~~: ReLU:

$$f(x) = \max(0, x)$$

Range: $(0, \infty)$

$$\text{Derivative} \rightarrow \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

There's no vanishing gradient when $x > 0$ (derivative = 1). This helps the training of deep networks. But if the activations are negative, derivative goes to 0, neurons will stop learning. Exploding gradients can happen if many positive derivatives multiply with large weights.

~~tanh~~:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Range: $(-1, 1)$

Derivative: $1 - \tanh^2(x) \in (0, 1)$ [maximum = 1]

This activation function will lead the values to be zero-centred, which can be good for optimization.

There's a chance of vanishing gradient when $|x|$ is large, saturates at -1 or +1. Exploding gradients are less common but possible when weights are large.

6. If $x = [2, 4, 6, 8, 10]$, $\gamma = 2$ and $\beta = -1$, perform batchnorm of all these 4 samples & write the output.

$$\tilde{z} = \gamma \hat{z} + \beta$$

$$\hat{z} = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad \text{or} \quad \frac{x_i - \mu}{\sigma + \epsilon}$$

$$\gamma = 2$$

$$\beta = -1$$

$$\mu = \frac{2+4+6+8+10}{5} = 6$$

Batch variance

$$\sigma^2 = \frac{(2-6)^2 + (4-6)^2 + (6-6)^2 + (8-6)^2 + (10-6)^2}{5}$$

$$= \frac{16 + 4 + 0 + 4 + 16}{5} = 8 \quad \sqrt{\sigma^2} = 2\sqrt{2} = 2.828$$

$$\hat{z} = \frac{x_i - \mu}{\sigma + \epsilon}$$

$$2 \Rightarrow \frac{2-6}{2.828} = -1.414$$

$$\hat{z} = -1.414, -0.707, 0, 0.707, 1.414$$

$$4 \Rightarrow \frac{4-6}{2.828} = -0.707$$

$$6 \Rightarrow 0$$

$$8 \Rightarrow 0.707$$

$$10 \Rightarrow 1.414$$

$$y/\tilde{z} = \gamma \hat{z} + \beta$$

$$\text{for } y_1 = 2(-1.414) + (-1) = -1.828$$

$$y_2 = 2(-0.707) + (-1) = -0.414$$

$$y_3 = 2(0) + (-1) = -1$$

$$y_4 = 2(0.707) + (-1) = 0.414$$

$$y_5 = 2(1.414) + (-1) = 1.828$$

After batchnorm, $\tilde{z}$ or y

$$\tilde{z} \text{ or } y = [-1.828, -0.414, -1, 0.414, 1.828]$$

7. What is the role of "keep probability" in the context of dropout and why dealing the activations with "keep probabilities" is necessary?

Role of keep probability 'p' $\rightarrow$ probability that a neuron stays active during training, where it directly controls the strength of regularization.

eg: 0.8 / 0.9 as dropout value $\rightarrow$ weak regularization.

By randomly keeping only a fraction of neurons each forward pass, the network is forced to learn more robust features instead of relying on specific neurons. solves co-adaptation.

If we don't scale, the expected activations will shrink by a factor of p. Ideally ensures that the expected activation stays the same as without dropout, so the model behaves consistently during training and testing. We scale the activations by $1/p$.

[for each neuron we multiply by $1/p$ during the forward pass] in inverted dropout.

8. For the following image and a filter, apply the convolution operation and write the output image. What did the convolution operation achieve in this example?

Input image

6x6

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

Filter

1	0	-1
1	0	-1
1	0	-1

3x3

6x6

$\rightarrow$ Output image size $(n-f+1) \times (n-f+1)$

$\rightarrow (6-3+1) \times (6-3+1)$

$\rightarrow 4 \times 4$

4) First row
 $\Rightarrow 10 + 0 - 10 + 10 + 0 - 10 + 10 + 0 - 10 \Rightarrow 0$

$\Rightarrow 10 + 0 - 0 + 10 + 10 \Rightarrow 30$

$\Rightarrow 10 + 10 + 10 \Rightarrow 30$

$\Rightarrow 0$

Output image

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

Second row

$\Rightarrow 0$

$\Rightarrow 30$

$\Rightarrow 30$

$\Rightarrow 0$

Third row

$\Rightarrow 0$

$\Rightarrow 30$

$\Rightarrow 30$

$\Rightarrow 0$

Fourth row

$\Rightarrow 0$

$\Rightarrow 30$

$\Rightarrow 30$

$\Rightarrow 0$

4x4

or edge

The convolution operation effectively highlights the boundary between the two regions [the transition from 10-10-10 to 0-0-0 in the input image], so the vertical edge was detected, as the output image shows a high-intensity line where the vertical edge is located and zeros elsewhere.

9. What is the impact of increasing the number of convolutional layers in CNN?

i) Hierarchical features are extracted $\Rightarrow$ we know that the first few layers in the deep CNN will capture the basic or low-level features such as edges, boundaries, corners etc. $\&$; intermediate layers will learn higher-level patterns such as shapes, objects parts etc. Deeper layers will capture semantic features like entire objects or complex features.

ii) Increased representational power: As the deeper layers will learn more abstract and complex patterns by combining the features from the preceding layers.

iii) Increased complexity - Computationally expensive and requires a large dataset to avoid overfitting

iv) Difficulty in optimization - Deeper networks have more complexity, hence it is harder for the optimizers to find a global minimum.

v) Vanishing / Exploding gradient problem - As the no. of layers increase, the gradients can either shrink to 0 or grow exponentially. This makes the learning ineffective.

10.

Discuss weight sharing and sparsity of connections in a CNN.

Weight sharing:

A single filter is applied across the entire input image. Instead of learning a separate weight for every connection pixel, the filter slides over the full image [same set of weights are shared across all locations in an image]. This drastically reduces the no. of parameters the network needs to learn, regardless of the image size. eg: a 3×3 filter applied to input image of any size has only 9 weights.

This reduces memory and computation power. Also helps us achieve **translational invariance** [recognize features of the image regardless of their position].

Sparsity of connections:

In a FC layer, every neuron is connected to all the inputs. eg: If an input image is $32 \times 32 = 1024$ px, and FC layer has 100 neurons that will be 1024×100 connections = 1,02,400 (dense).

In CNN $\rightarrow$ each neuron is connected only to a small localized region of the input which is called **receptive field**. eg: If we use a 3×3 filter $\rightarrow$ each neuron "sees" only 9 pixels at a time, not the entire input image. So neurons are ^{not} connected to every input pixel. Just only a small subset, so the connections are sparse. This will reduce the no. of parameters and makes it computationally efficient.

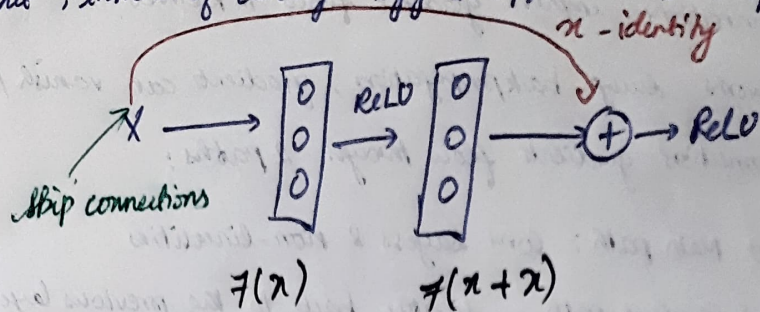
11.

How does ResNet work and show how it can easily learn the identity function?

A ResNet ~~convnet~~ architecture consists of residual blocks where the residual blocks contains two or more convolutional layers, where the output of the block is obtained by adding the input of the block to the output of the last layer in the block.

This allows the network to learn residual representations, meaning that it learns the difference between the input and the desired

output, instead of directly trying to approximate the output.



Mathematical representation: $\rightarrow H(x) = f(x) + x$

where x - input; $f(x)$ - residual function; $H(x)$ - mapping function from input to output.

ResNet solves the vanishing gradient problem in deep NN by introducing skip connections. For the above figure, the activation of the current layer would be,

$$a^{[l+2]} = \left(\text{ReLU}(w^{[l+2]} a^{[l+1]} + b^{[l+2]} + a^{[l]}) \right)$$

$\Rightarrow$ When the same gradient is coming from two paths through the current layer and the previous layer $\rightarrow$ we will add them which will avoid the vanishing gradient problem.

$\Rightarrow$ The ResNet learns the identity function, by simply making $f(x)$ as 0.

$$H(x) = f(x) + x$$

$$x = f(x) + x$$

$$f(x) = 0$$

As the ResNet block is calculated as the sum of input and the output of its convolutional layers.

To learn the function which makes the output $f(x)$ zero is simpler for a neural network rather than making $f(x) = x$.

This can be achieved by setting the weights and biases to very small values, effectively "turning off" the computations.

The skip connection $+x$ handles the important job of passing the input directly through and the network only needs to learn minor adjustments $f(x)$ if the identity mapping is not perfect. This simple goal of learning 0 makes ResNets much easier to train and much effective at building very deep architectures.

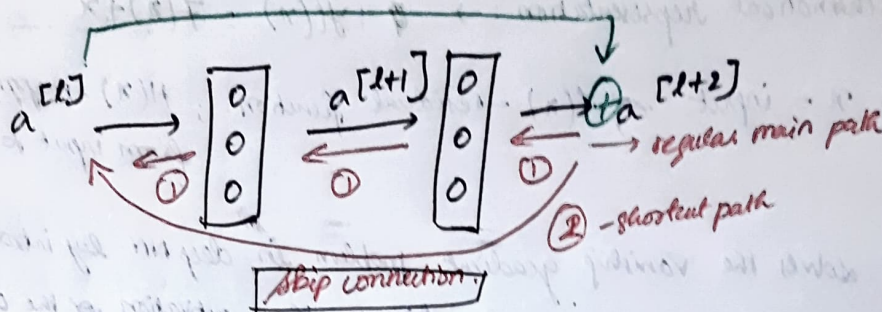
12. How do skip connections improve gradient flow in ResNet?

⇒ In deep networks, during backpropagation, gradients can vanish/explode.

⇒ In skip connections gradients flow through 2 paths:

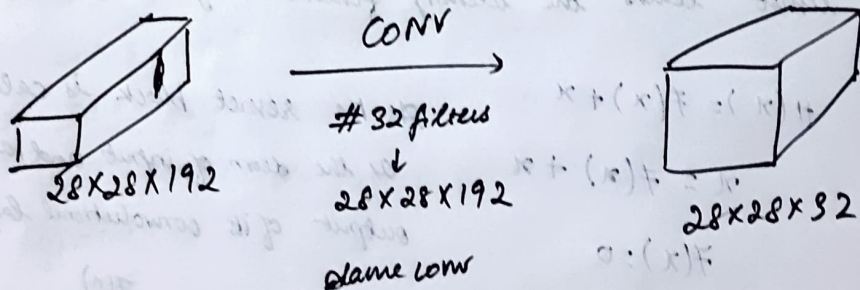
⇒ Main path: conv layers & non-linearities

⇒ Shortcut path, directly back to the previous layer



This ensures that even if gradients shrink in the main path, the shortcut path provides an unimpeded gradient flow (bypass/supervising).

13. For the following CONV operation, compute the computational cost [no. of multiplications for example]



$$28 \times 28 \times 192 = 150528$$

$$\text{output} \rightarrow \text{same convolution} = 150528 \times 28 \times 28 = 118013752 \rightarrow \text{for 1 filter}$$

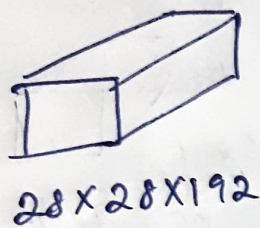
$$32 \text{ filters} \rightarrow \text{Total} = 118013752 \times 32$$

$$= 3776446464$$

$$\approx 3.7 \text{ billion computations / multiplications}$$

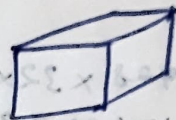
13. continued...

Design a new network architecture using 1×1 conv layers to reduce the computational cost. Show the savings from your architecture. Please note that your new proposed architecture should take the same sized input image ($28 \times 28 \times 192$) and output the image of the same size shown in the above diagram ($28 \times 28 \times 32$).



input

1×1 conv
32 filters



output

by applying 1×1 conv, our computational cost would be,

$$1 \times 1 \times 192 \rightarrow 192 \times 28 \times 28 \rightarrow \text{for 1 filter}$$

$$= 150528 \text{ computations}$$

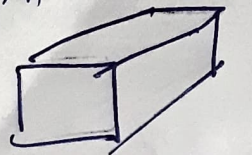
$$\text{for 32 filters} \rightarrow 150528 \times 32$$

$$= 4816896 \text{ computations}$$

$$\text{savings in our architecture} = \frac{3776446464 - 4816896}{3776446464} \times 100\%$$

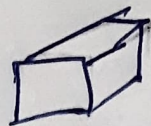
$$= 99.877\% \text{ fewer computations}$$

13. → Alternate architecture



$28 \times 28 \times 192$

$\xrightarrow{1 \times 1 \text{ conv}}$
64 filters



$28 \times 28 \times 64$

$\xrightarrow{3 \times 3 \text{ conv}}$
32 filters



$28 \times 28 \times 32$

First layer

$$1 \times 1 \times 192 = 192$$

$$28 \times 28 \times 192 = 150528 \text{ for 1 filter}$$

$$64 \text{ filters} \rightarrow 9633792$$

Second layer

$$3 \times 3 \times 64 = 576$$

$$576 \times 28 \times 28 \rightarrow 1 \text{ filter}$$

$$32 \text{ filters} \rightarrow 14450688$$

$$\text{Add both comput}^n \text{ from 2 layers} \Rightarrow 24084480$$

$$\text{Savings} \Rightarrow \frac{3776446464 - 24084480}{3776446464} = 99.36\%$$

9633792

14.

I would like to construct an inception module with 128 3×3 conv filters, 32 5×5 conv filters, same convolution, stride=1. The input to the inception module (coming from previous layer output activation) size is $32 \times 32 \times 128$ and the output image is $32 \times 32 \times 160$.

a) What will be the no. of multiplications in the above inception module.

b) Provide a strategy to reduce the no. of multiplications and write down

the no. of multiplications after adopting your strategy.



$32 \times 32 \times 128$

Firstly 128 filters

3×3 CONV



$32 \times 32 \times 128$

$$1 \text{ filter} = 3 \times 3 \times 128 = 1152 \times 32 \times 32 \times 128$$

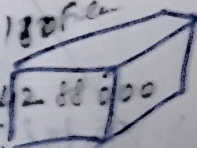
$$1152 \times 128 = 147456 \times 128 = 18874304$$

3×5 CONV

Semi conv.

160 filters

Stride = 1



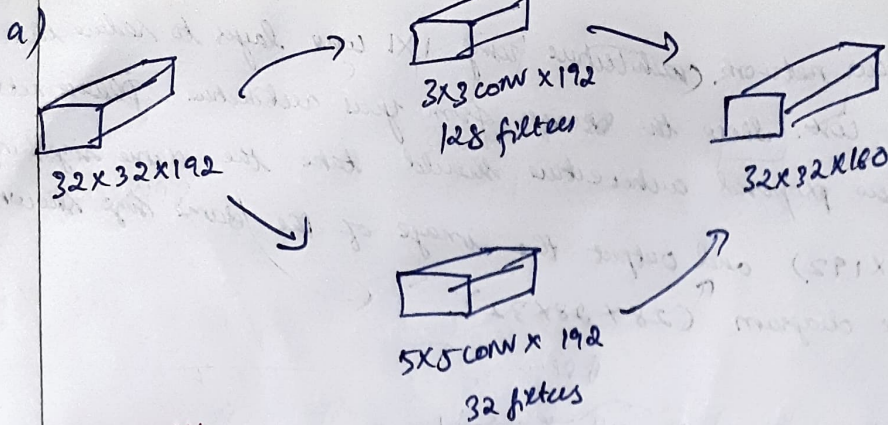
$32 \times 32 \times 160$

2nd layer

$$5 \times 5 \times 128 = 3200$$

$$3200 \times 32 \times 32 = 3276800$$

$$3276800 - 160 = 3276640$$



3x3 conv:

$$3 \times 3 \times 192 = 1728 \times 32 \times 32$$

$$= 1769472 \rightarrow \text{for 1 filter}$$

$$\rightarrow 226492416 \rightarrow \text{for 128 filters}$$

5x5 conv:

$$5 \times 5 \times 192 \rightarrow 4800 \times 32 \times 32$$

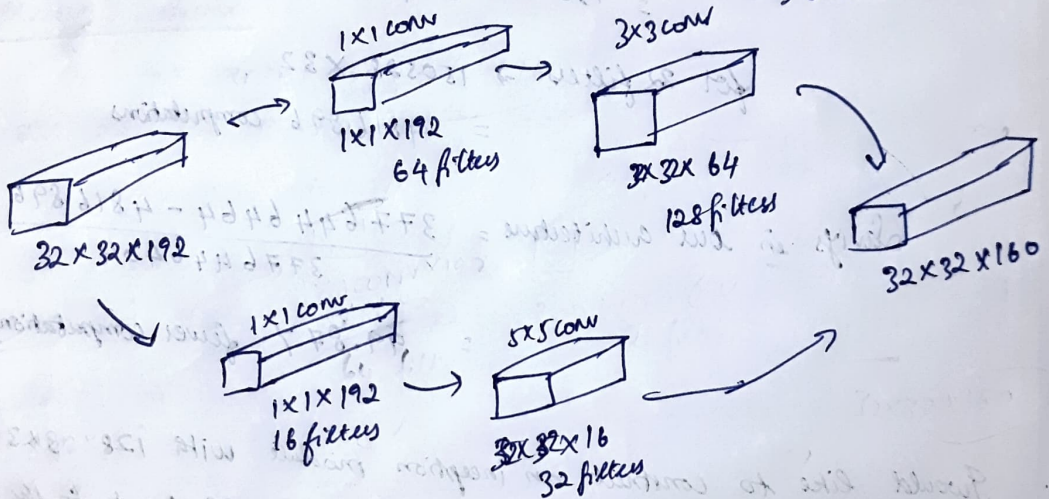
$$= 4915200 \rightarrow \text{1 filter.}$$

$$157286400 \rightarrow \text{for 32 filters}$$

$$\text{Total} \Rightarrow 157286400 + 226492416 = 383778816 \text{ computations.}$$

= 3.8 billion computations

b)



1x1 conv $\rightarrow$ 3x3 conv

$$1 \times 1 \text{ conv} \Rightarrow 192 \times 64 = 12288 \times 32 \times 32 = 12582912$$

$$3 \times 3 \text{ conv} \Rightarrow 3 \times 3 \times 64 \Rightarrow 576 \times 32 \times 32 \Rightarrow 589824 \times 128 = 75497472$$

$$\text{Total: } 88080384$$

1x1 conv $\rightarrow$ 5x5 conv

$$1 \times 1 \text{ conv} \Rightarrow 192 \times 16 = 3072 \times 32 \times 32 = 3145728$$

$$5 \times 5 \text{ conv} \Rightarrow 5 \times 5 \times 16 = 400 \Rightarrow 400 \times 32 \times 32 = 409600 \rightarrow \text{1 filter}$$

$$32 \text{ filters} \rightarrow 13107200$$

$$\text{Total} = 16252928$$

$$\text{Total: } 88080384 + 16252928 = 104333312 \text{ computations}$$

Savings:

$$\frac{383778816 - 104333312}{383778816} = \frac{279445504}{383,778,816} = 0.728 \times 100 \Rightarrow 72.8\%$$

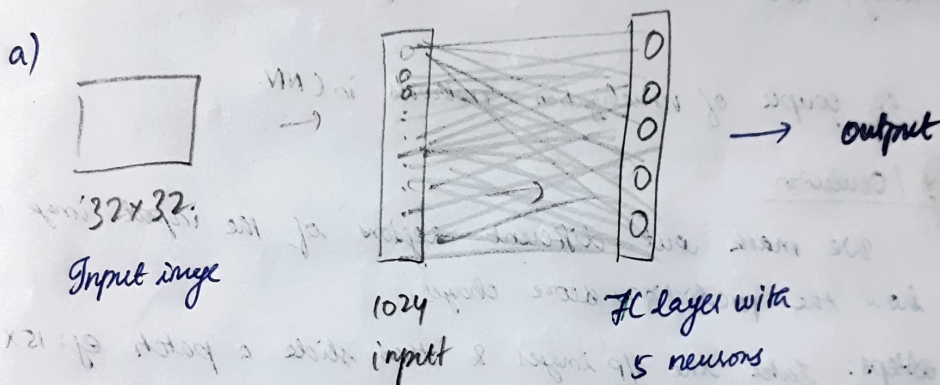
72.8% fewer ~~data~~ computations

15.

You have to develop a 5-way clf to detect objects in an image. You are given a grayscale input image of size 32×32 . Which of the following approaches is better and why?

a) Flatten the image (32×32) to form a 1024 vector and pass this on to a FC layer with 5 neurons

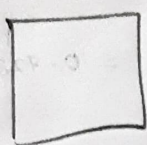
b) Directly pass the image to CNN layer having filter 32×32



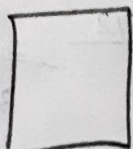
⇒ An FC layer with 1024 input neurons & 5 output neurons (direct classification) would have a large no. of weights $1024 \times 5 = 5120$ which can be computationally expensive → Too many parameters if we expect to deeper layers which is prone to overfitting on smaller datasets.

⇒ As we know in this FC approach, every pixel is treated independently, even though they have crucial spatial relationships between the pixels. So the features of the image are not learnt properly.

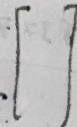
⇒ This method is not at all generalizable, meaning if the object shifts even slightly in the image, the input vector changes dramatically making it difficult for the network to perform well on new/unseen images.



32x32



32x32



1x1x5

5 filters

Here the no. of parameters are the same as the previous option $32 \times 32 \times 5 = 5120$. The only difference between both the approaches is the weight sharing $\rightarrow$ But as the filter covers the whole image it behaves almost same as the FC. (no sliding)
Essentially, this network is not really doing the local features.

16. Explain a couple of visualization methods in CNN.

Masking / Occlusion.

We mark out different regions of the input image and see how the prediction score changes.

Steps: Take the i/p images & then slide a patch eg: 15×15 over the image, at each location, replace that patch with a black pixel ($\begin{smallmatrix} 0 \\ 0 \end{smallmatrix}$); record how the prediction confidence for the target class changes.

t-SNE:

A dimensionality reduction technique used to visualize high-dim data (like CNN embeddings) in 2D or 3D.

CNNs' output $\rightarrow$ feature vectors. projects them into 2D so that we can visualize the structure. [Points that are close together in high-dim space & stay close in 2D. Points that are far apart remains separated]

We can use this on trained CNN $\rightarrow$ pass images of different classes $\rightarrow$ extract embeddings $\rightarrow$ apply t-SNE to reduce embeddings to 2D $\rightarrow$ Plot with points colored by class label.